# WEEK 5
# BEGINNING PYTHON
# STUDY GUIDE

Modules, Required Deliverables, Practice Questions & Final Glossary

| | |
|---|---|
| **Modules** | Introduction to Python; Building Expressions; Working with Data Types; Conditional Logic & Flow Control |
| **Primary tools** | Visual Studio Code, Python extension, Git, GitHub, Terminal, and optional Jupyter Notebook |
| **Submission folder** | **DataAnalytics/week-05** |

**How to study:** Read one section, cover the notes, answer the questions, then type the sample code from memory. A Python concept is mastered only when you can predict the result before running the file.

Prepared from the Week 5 Lab Workbook

# Week 5 Deliverables at a Glance

Required outputs are emphasized below. Bonus items are marked optional.

| Exercise | Folder / File | Purpose |
|---|---|---|
| 1.A | Ex1A_hello.py | First program, variables, comments, numeric addition, string concatenation |
| 1.B | Ex1B_variable_examples.py | Good variable names, values, assumptions, reserved-word research |
| 1.B Bonus | Ex1B_Notebook_Exploration.ipynb | Optional Jupyter Notebook exploration |
| 2.A | Ex2A_tips.py | Calculations, str(), format(), and printed output |
| 2.A folder | Ex2A_MathScripts | Contains net worth, rectangle, tip, circle, Rule of 72 scripts |
| 2.B folder | Ex2B_MoreMathScripts | Temperature, taxes, distance, tiles, and rentals scripts |
| 3.A | Ex3A_conversion_tests.py | Casting, slicing, strip(), type checking, error documentation |
| 3.A | Ex3A_string_cleaning.py | lower(), title(), replace(), chained conversion |
| 3.B folder | Ex3B_SampleCollections | List, tuple, set, and dictionary exercises |
| 4.A folder | Ex4A_IfScripts | Basic conditionals, overtime pay, optional leap-year script |
| 4.B folder | Ex4B_MoreIfScripts | Lookups, greetings, taxes, min/max, and major lookup |
| 4.C folder | Ex4C_LoopScripts | for, while, enumerate, reversed, tuple unpacking |
| 4.D Bonus | Ex4D_Bonus_number_guesser.py | Optional number-guessing game |

**Submission routine:** Save, run, inspect output, comment observations, commit with a descriptive message, sync/push, and confirm the latest file in GitHub.

# Module 1 - Introduction to Python

## Important things to study

- Opening DataAnalytics/week-05 in Visual Studio Code
- Using the Python extension, Run button, Terminal, Explorer, and Source Control
- Creating and running .py files
- Variables, assignment, comments, print(), strings, and numbers
- Python naming rules, snake_case, and reserved words
- Saving, committing, syncing, and verifying work

**Variables:** A variable name points to a value. Assignment uses =, such as `message = "Hello world!"`. The value can later be replaced.

**Naming:** A variable must begin with a letter or underscore, cannot use a reserved word, and should normally use lowercase snake_case. Names should explain the value being stored.

**Comments:** A line beginning with # is ignored by Python and documents purpose, reasoning, assumptions, or observations.

```
message = 'Hello world!' print(message) dollars = 3 cents = 0.50 print(dollars + cents)
```

## Required deliverables

**Ex1A_hello.py**
Print Hello world, create message, dollars, cents, d_str, and c_str variables, update cents, combine strings, and add explanatory comments.

**Ex1B_variable_examples.py**
Create well-named customer and personal variables, record assumptions as inline comments, list reserved words, and define five selected keywords.

**Optional: Ex1B_Notebook_Exploration.ipynb**
Run code in notebook cells, print a variable, save locally, then commit and push.

## Questions to answer

**1.** Why does Hello world print twice after the message variable is added?

**2.** What is the difference between adding two numbers and concatenating two strings?

**3.** Why would customer_id be a better name than id1?

**4.** What happens when a variable is assigned a new value?

**5.** What is the purpose of an inline comment?

**6.** Why must reserved words not be used as variable names?

# Module 2 - Building Expressions

## Important things to study

- Turning a word problem into known values, a formula, and code
- Arithmetic operators and order of operations
- Converting values with str()
- Formatting decimals and percentages with format()
- Getting user data with input()
- Writing f-strings
- Testing calculations with independent examples

**Expression:** A combination of values, variables, and operators that Python evaluates to produce a result.

**Type conversion:** str() converts a value to text. int() and float() convert compatible values to numeric types. input() always returns a string unless you convert it.

**Formatting:** format(value, ".2f") displays two decimal places; format(rate, ".0%") displays a decimal as a percentage. Formatting changes the display, not the underlying calculation.

**F-string:** Begins with f and places variables or expressions inside curly braces. It avoids manual string concatenation and repeated str() calls.

```
food_cost = 79.25 tax = 6.54 tip = 12.00 total_due = food_cost + tax + tip print(f'The total due is {total_due:.2f}')
```

## Core formulas to know

- Net worth = total assets - total debts
- Rectangle area = length x width
- Tip amount = bill x tip rate
- Circle area = pi x radius squared
- Rule of 72 years = 72 / interest rate as a whole percentage
- Fahrenheit to Celsius = (F - 32) x 5 / 9
- Celsius to Fahrenheit = (C x 9 / 5) + 32
- Distance = square root of ((x2-x1)^2 + (y2-y1)^2)
- Full boxes needed require rounding up, not ordinary rounding

# Module 2 - Deliverables & Review

## Required deliverables

| |
|---|
| **Ex2A_tips.py**<br>Calculate food cost, tax, tip, and total. Use str(), format(), and clear output. |
| **Ex2A_MathScripts/net_worth.py**<br>Report total assets, debts, and net worth. |
| **Ex2A_MathScripts/area_of_rectangle.py**<br>Calculate area from two sides. |
| **Ex2A_MathScripts/tip_amount.py**<br>Calculate a tip from a bill and rate. |
| **Ex2A_MathScripts/area_of_circle.py**<br>Convert diameter to radius and calculate area. |
| **Ex2A_MathScripts/rule_of_72.py**<br>Calculate doubling time and format balance, years, and percentage. |
| **Ex2B_MoreMathScripts**<br>Create f_to_c.py, c_to_f.py, taxes.py, distance.py, tiles.py, and rentals.py. |

## Questions to answer

**1.** Why does input() create a string even if the user enters 25?

**2.** When should you use int() instead of float()?

**3.** What is the difference between round() and format()?

**4.** Why must the tile and van problems round up?

**5.** Rewrite print("Total is " + str(total)) as an f-string.

**6.** What test cases would you use for a conversion script?

**7.** Why can a correct-looking formula still produce a wrong result?

# Module 3 - Working with Data Types

## Important things to study

- Casting strings to int and float
- Using type() to inspect values
- Indexing and slicing strings
- Cleaning strings with strip(), lower(), upper(), title(), and replace()
- Understanding lists, tuples, sets, and dictionaries
- Sorting, appending, indexing, and updating collections

**Casting:** int("55") and float("101.1") work because the strings contain compatible numeric text. int("101.1") fails, but int(float("101.1")) works by converting in two steps.

**String cleaning:** strip() removes leading and trailing whitespace; replace() substitutes characters; title(), lower(), and upper() modify capitalization. Methods return cleaned values but do not automatically replace the original variable.

**Slicing:** Uses zero-based indexes and a stop position that is not included. Example: text[0:3] returns positions 0, 1, and 2.

```
salary = '$82,500' clean_salary = int(salary.replace('$', '').replace(',', ''))
print(clean_salary, type(clean_salary))
```

## Collections at a glance

| Type | Ordered? | Duplicates? | Mutable? | Best use |
|---|---|---|---|---|
| List | Yes | Yes | Yes | Sequence that may change |
| Tuple | Yes | Yes | No | Fixed record or sequence |
| Set | No stable display order | No | Yes | Unique values |
| Dictionary | Insertion ordered | Keys unique | Yes | Key:value lookup |

# Module 3 - Deliverables & Review

## Required deliverables

| |
|---|
| **Ex3A_conversion_tests.py**<br>Test int(), float(), int(float()), slicing, strip(), print(), and type(). Comment out failing lines and document the error type. |
| **Ex3A_string_cleaning.py**<br>Clean inconsistent names and salary strings; chain replace() and int() to make salary numeric. |
| **Ex3B_SampleCollections/movie_list.py**<br>Create and measure a list; compare sorted() with .sort(); append a movie. |
| **Ex3B_SampleCollections/candy_store.py**<br>Create tuples and a set of candy combinations; observe set output order. |
| **Ex3B_SampleCollections/address_entry.py**<br>Create, remove, nest, and update dictionary key:value pairs; print a mailing address. |

## Questions to answer

**1.** Why does int("101.1") fail while int(float("101.1")) succeeds?

**2.** What is the difference between sorted(my_list) and my_list.sort()?

**3.** Why might a set print in a different order?

**4.** When is a tuple more appropriate than a list?

**5.** What is a dictionary key:value pair?

**6.** Why does removing $ from a salary string not automatically make it numeric?

**7.** What does type() help you diagnose?

# Module 4 - Conditional Logic

## Important things to study

- Boolean expressions and comparison operators
- if, elif, and else order
- Nested and multi-condition logic
- match/case for discrete lookup values
- Boundary testing
- Gross pay, overtime, taxes, greetings, minimum, maximum, and lookup scripts

**Conditional:** Runs a block only when its Boolean test is True. Indentation defines which statements belong to each branch.

**Order matters:** Python checks if/elif branches from top to bottom and runs the first matching branch. Put narrow or special cases before broader cases.

**Boundaries:** Test values just below, exactly at, and just above each threshold. For overtime, test 39, 40, and 41 hours.

```
if hours_worked > 40: regular_pay = pay_rate * 40 overtime_pay = (hours_worked - 40) * pay_rate * 1.5 gross_pay = regular_pay + overtime_pay else: gross_pay = hours_worked * pay_rate
```

## Required deliverables

**Ex4A_IfScripts**
Create if_basics.py and pay_rules.py; optional bonus_gregorian_calendar.py.

**Ex4B_MoreIfScripts**
Create dept_converter.py, dept_converter_v2.py, greeting.py, complex_taxes.py, min_max.py, and show_major.py.

## Questions to answer

**1.** Why is indentation part of Python syntax?

**2.** What happens when more than one condition could be true?

**3.** Why should exactly 40 hours be tested separately from values above and below 40?

**4.** When is match/case easier to read than if/elif/else?

**5.** How would you handle an unknown department or major code?

**6.** Why should tax logic be tested near every income threshold?

# Module 4 - Loops & Flow Control

## Important things to study

- for loops for iterating through known collections
- while loops for repeating until a condition changes
- Avoiding infinite loops
- enumerate() for indexes and values
- reversed() for reverse iteration
- Tuple unpacking in a loop
- Accumulators and conditions inside loops

**for loop:** Best when iterating over a collection or known sequence. The loop variable takes one item at a time.

**while loop:** Repeats while a Boolean condition remains True. The loop body must eventually change something that can make the condition False.

**enumerate():** Produces an index and item together. Use start=1 when displaying a human-friendly numbered list.

**Accumulator:** A variable initialized before the loop and updated during each iteration, such as total_sales += sales.

```
for rank, item in enumerate(items, start=1): print(f'{rank}. {item}') total_sales = 0 for name, region, sales in sales_data: total_sales += sales
```

## Required deliverables

**Ex4C_LoopScripts/loop_love.py**
Loop through descriptive words and print a sentence for each.

**Ex4C_LoopScripts/savings_goal.py**
Use a while loop, progress messages, and optional treat logic.

**Ex4C_LoopScripts/ranked_list.py**
Use enumerate(), a first-item condition, and optional reversed().

**Ex4C_LoopScripts/sales_performance.py**
Unpack tuples, format currency, flag top performers, and optionally accumulate total sales.

**Optional Ex4D_Bonus_number_guesser.py**
Build a number pool, accept guesses, give higher/lower hints, track guesses, and handle invalid input.

## Questions to answer

**1.** When should you use a for loop instead of a while loop?

**2.** What causes an infinite while loop?

**3.** How does enumerate(items, start=1) differ from a plain for loop?

**4.** What is tuple unpacking?

**5.** Where should an accumulator be initialized?

**6.** Why is the order of conditions inside savings_goal.py important?

**7.** How would you make number-guesser input safe for non-numeric text?

# Python Quick Reference

## Core syntax

```
# Comment variable_name = value print(variable_name) user_text = input('Enter a value: ') number = float(user_text) print(f'Result: {number:.2f}')
```

```
if condition: # indented block elif other_condition: # another block else: # fallback block for item in collection: print(item) while condition: # update condition-related value
```

## Common failure modes

- TypeError: combining incompatible types, such as text and a number, without conversion.
- ValueError: converting text that is not a valid number.
- SyntaxError: invalid Python structure, missing punctuation, or malformed statement.
- IndentationError: code blocks are not indented consistently.
- IndexError: list or tuple index is outside the valid range.
- KeyError: dictionary key does not exist.
- Infinite loop: while condition never becomes False.
- Wrong result: formula, condition order, boundary, or data type is incorrect even though the script runs.

**Debugging routine:** Read the final error line, note the exception type and line number, inspect the variable values and types, reduce the problem to one expression, correct it, and rerun. Preserve required failing examples as comments when the lab asks you to document them.

# Final Completion Checklist

Check each item only after the file runs and the latest version is visible in GitHub.

[ ] All required files and folders use the exact requested names.

[ ] Every script is saved under DataAnalytics/week-05.

[ ] Every required script runs without unexpected errors.

[ ] Required comments explain observations, assumptions, or errors.

[ ] Calculations were tested with more than one input.

[ ] Conditional scripts include boundary and fallback tests.

[ ] Loop scripts terminate and produce the intended output.

[ ] Source Control shows the intended changes.

[ ] Changes are committed with descriptive messages.

[ ] Sync/push completed and GitHub shows the latest files.

## Mixed review questions

**1.** What type does input() return, and why does that matter?

**2.** Explain the difference between a list, tuple, set, and dictionary.

**3.** Predict the output of a short script that reassigns a variable.

**4.** Why does sorted() not permanently change the original list?

**5.** Write an if/elif/else structure for morning, day, and evening greetings.

**6.** Write a for loop that prints each item in a list.

**7.** Write a while loop that increases balance until a goal is met.

**8.** Name three error types and the situation that causes each.

# Final Glossary - Terms to Memorize & Master

Cover the definitions and explain each term aloud from memory.

| | |
|---|---|
| **Python**: General-purpose programming language. | **Script**: File containing executable code. |
| **IDE**: Integrated environment for writing and running code. | **VS Code**: Editor used to create and run Week 5 scripts. |
| **Terminal**: Text interface where commands, prompts, and output appear. | **Repository**: Version-controlled project folder. |
| **Commit**: Saved version-control snapshot. | **Sync / Push**: Publishes local commits to GitHub. |
| **Variable**: Name that refers to a value. | **Assignment**: Storing a value with =. |
| **Identifier**: Valid name for a variable or object. | **snake_case**: Lowercase words joined by underscores. |
| **Reserved word**: Python keyword unavailable as a variable name. | **Comment**: Non-executed documentation beginning with #. |
| **print()**: Displays a value or message. | **Expression**: Values, variables, and operators evaluated together. |
| **Operator**: Symbol performing arithmetic or comparison. | **String**: Sequence of text characters. |
| **Integer**: Whole-number type. | **Float**: Decimal-number type. |
| **Boolean**: True or False value. | **Casting**: Converting a value from one type to another. |
| **str()**: Converts a value to a string. | **int()**: Converts compatible data to an integer. |
| **float()**: Converts compatible data to a float. | **type()**: Returns the type of a value. |
| **input()**: Prompts the user and returns a string. | **format()**: Formats a value for display. |
| **f-string**: Formatted string with expressions in braces. | **Module**: File or library containing reusable Python code. |
| **Index**: Zero-based position in a sequence. | **Slice**: Selected portion of a sequence. |
| **Method**: Function attached to an object. | **strip()**: Removes leading and trailing whitespace. |
| **replace()**: Substitutes text within a string. | **lower()**: Returns lowercase text. |
| **title()**: Capitalizes the first letter of each word. | **List**: Ordered, mutable collection. |
| **Tuple**: Ordered, immutable collection. | **Set**: Collection of unique values. |
| **Dictionary**: Mutable key:value mapping. | **len()**: Returns the number of items. |
| **sorted()**: Returns a new sorted collection. | **.sort()**: Sorts a list in place. |
| **.append()**: Adds one item to a list. | **.update()**: Adds or changes dictionary entries. |
| **Conditional**: Code that runs based on a Boolean test. | **if**: First conditional branch. |
| **elif**: Additional conditional branch. | **else**: Fallback branch. |
| **match/case**: Pattern-based branching for discrete values. | **Comparison operator**: Compares values, such as ==, <, or >. |
| **Indentation**: Whitespace defining Python code blocks. | **Boundary test**: Test at, below, and above a threshold. |
| **for loop**: Iterates through a collection or sequence. | **while loop**: Repeats while a condition is True. |
| **Iteration**: One pass through a loop. | **Infinite loop**: Loop that never reaches a stopping condition. |
| **enumerate()**: Returns index and item pairs. | **reversed()**: Iterates through a sequence in reverse. |
| **Tuple unpacking**: Assigns tuple elements to separate variables. | **Accumulator**: Variable updated across loop iterations. |
| **SyntaxError**: Invalid Python structure. | **TypeError**: Operation uses incompatible types. |
| **ValueError**: Right type of input, but invalid value. | **IndexError**: Sequence position is out of range. |
| **KeyError**: Dictionary key is missing. | **Algorithm**: Step-by-step method for solving a problem. |
| **Test case**: Inputs and expected output used to verify code. | : |